

Guía de estudio y sustentación — Agente Doom (YOLO + Double DQN)

Documento para leer y estudiar antes de sustentar. Va de lo simple a lo técnico. Cada sección tiene primero la **idea intuitiva** y luego la **matemática**, para que puedas explicarlo con tus palabras y defender la fórmula si te preguntan.

Relacionado: sustentacion_proyecto_doom (el PDF de diapositivas) · notebook `doom_entrega.ipynb`

0. La idea en 30 segundos (tu frase de apertura)

"Construimos un agente que juega Doom. En vez de aprender directamente desde los píxeles, separamos el problema en dos: primero un **detector YOLO** mira el frame y dice qué enemigos hay y dónde (la percepción), y luego un **agente de aprendizaje por refuerzo Double DQN** decide qué hacer con esa información (la política). Esto hace el aprendizaje mucho más eficiente y, sobre todo, interpretable."

Si solo memorizas un párrafo, que sea ese. Todo lo demás lo desarrolla.

1. Mapa mental del sistema

El flujo de **un paso de juego** es una cadena:

```
ViZDoom → frame RGB → YOLO → cajas de enemigos → 13 features → FrameStack(×3) → 39 números  
→ Double DQN → acción (girar/avanzar/disparar) → se ejecuta en ViZDoom → recompensa r  
→ (vuelve a empezar)
```

Dos mitades, dos responsables:

MITAD	QUÉ HACE	RESPONSABLE
Percepción	Convierte el frame en información semántica	David
Política (control)	Decide la acción a partir de esa información	Joshua

La pregunta clave que defiende todo el diseño: **¿por qué no aprender directo de los píxeles?** Porque un frame es $240 \times 320 \times 3 \approx 230.000$ números. Un DQN sobre eso tendría que aprender a "ver" y a "decidir" al mismo tiempo, lo que exige millones de partidas y una GPU enorme. Al meter YOLO en medio, el

agente solo aprende sobre **39 números con significado**, no sobre un mar de píxeles. Más rápido y explicable.

2. La percepción (YOLO)

Idea intuitiva

YOLO es un detector de objetos: le das una imagen y te devuelve cajas con "aquí hay un zombieman, aquí un pinky". Es el sistema visual del agente.

El truco clave: dataset in-domain auto-etiquetado

Problema que tuvimos: al principio el detector se entrenó con imágenes de Doom de internet (Roboflow), pero el agente jugaba en FreeDoom/ViZDoom, que se ven distinto. Esa **brecha de dominio** hacía que el detector "alucinara" enemigos en las paredes.

Solución: ViZDoom tiene un `labels_buffer`, un canal oculto donde el motor te dice exactamente qué objeto hay en cada píxel y su nombre. Entonces capturamos frames del propio juego y sacamos las cajas ground-truth **automáticamente, sin etiquetar a mano**. Resultado: el detector se entrena con imágenes idénticas a las que verá jugando.

Si te preguntan "¿cómo etiquetaron miles de imágenes?": la respuesta es que **no las etiquetamos a mano**, el motor del juego nos dio las etiquetas gratis vía `labels_buffer`.

Datos y resultado

- 2.213 imágenes (1.817 entrenamiento + 396 validación), de 4 escenarios.
- 20% de frames vacíos a propósito, para enseñar a no ver nada cuando no hay nada.
- Modelo final `doom-v4`: YOLOv8s, 100 épocas, 640 px.
- **Métricas:** Precisión 0.930 · Recall 0.783 · mAP@0.5 = 0.908 · mAP@0.5:0.95 = 0.779.

Por qué solo confiamos en 4 clases

La matriz de confusión mostró que solo `pinky` (0.96), `shotgun-guy` (0.92) y `zombieman` (0.83) se reconocen bien. `cacodemon` se confunde con fondo el 36% de las veces, y otras clases no tienen datos suficientes. Decisión honesta: **percepción estrecha pero fiable** es mejor que amplia pero ruidosa. Por eso solo esas 4 clases cuentan para la recompensa.

Vocabulario para definir

- **mAP@0.5:** promedio de la precisión de detección exigiendo que la caja predicha solape al menos 50% con la real ($\text{IoU} \geq 0.5$). Es la métrica estándar de detección.
 - **Precisión:** de lo que detecté, qué fracción era correcta. **Recall:** de lo que había, qué fracción detecté.
-

3. El estado (de cajas a 13 números)

Las cajas de YOLO más la vida y la munición se comprimen en un **vector de 13 features normalizadas** (todas entre 0 y 1, o entre -1 y 1). Las importantes:

- ¿Hay enemigo? (0/1)
- Offset horizontal del enemigo más centrado (-1 izquierda, +1 derecha)
- Tamaño del enemigo (proxy de cercanía)
- Vida y munición
- ¿Enemigo centrado? (listo para disparar)
- Distancia real al enemigo (leída del **depth buffer** de ViZDoom)

FrameStack: dándole memoria

Un solo frame no dice si el enemigo se acerca o se aleja. Por eso **apilamos los últimos 3 vectores**: $13 \times 3 = 39$ **dimensiones**. Eso le da al agente una memoria corta para percibir movimiento, sin necesidad de una red recurrente (LSTM).

Frase: "El estado final son 39 números: 13 features de percepción por cada uno de los últimos 3 frames, para capturar el movimiento."

4. La política: aprendizaje por refuerzo (lo central)

4.1 El marco: Proceso de Decisión de Markov (MDP)

El agente vive un ciclo: observa un **estado** s , toma una **acción** a , recibe una **recompensa** r y pasa a un nuevo estado s' . Su meta no es la recompensa inmediata sino el **retorno**: la suma de recompensas futuras, descontadas.

- **Descuento $\gamma = 0.99$** : una recompensa dentro de k pasos vale γ^k de su valor. Con 0.99 el agente es "paciente", valora el futuro casi como el presente. Necesario porque en Doom las recompensas (matar, llegar a la meta) llegan **después** de la secuencia de acciones que las causaron.

4.2 La función Q y la ecuación de Bellman

Definimos $Q(s,a)$ = "qué tan bueno es tomar la acción a en el estado s , y luego jugar óptimamente". La Q óptima cumple la **ecuación de Bellman**:

$$Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

En palabras: el valor de una acción = la recompensa que da ahora, más el mejor valor que puedo conseguir desde el estado al que me lleva (descontado). Si conozco Q , *mi política óptima es trivial: en cada estado, elige la acción con mayor Q^* .*

4.3 ¿Por qué una red neuronal?

No podemos guardar Q en una tabla porque el estado es continuo (39 números reales, infinitas combinaciones). Así que **aproximamos Q con una red neuronal** de parámetros θ . Eso es un "Deep Q-Network" (DQN). La red recibe el estado (39) y saca un valor Q por cada acción (13).

5. Las 4 mejoras sobre el DQN base (una por una)

Esta es la sección que más impresiona en sustentación, porque cada mejora **resuelve un problema concreto** del DQN original. Apréndete el "problema → solución" de cada una.

5.1 Double DQN — corrige la sobreestimación

Problema: el DQN usa el mismo `max` para *elegir* y *evaluar* la mejor acción siguiente. Eso hace que el ruido se sesgue siempre hacia arriba: sobreestima los valores.

Solución: usar dos redes. La red **online** (θ) elige cuál es la mejor acción; la red **target** (θ^- , una copia retrasada) dice cuánto vale. Separar elegir de evaluar quita el sesgo.

$$y = r + \gamma \cdot Q(s', \underset{\uparrow \text{elige con } \theta^+}{\operatorname{argmax}_a} Q(s', a'; \underset{\uparrow \text{evalúa con } \theta^-}{\theta}) ; \theta^-)$$

5.2 Dueling — aprende el valor del estado más rápido

Problema: en muchos estados, la acción concreta da casi igual (p. ej. si no hay enemigos, da lo mismo girar a un lado u otro). El DQN normal tiene que aprender el valor de cada acción por separado.

Solución: la red separa Q en dos partes: - **V(s)**: qué tan bueno es el estado en sí (¿estoy en peligro?). - **A(s,a)**: la ventaja de cada acción sobre el promedio.

$$Q(s,a) = V(s) + (A(s,a) - \text{media}_a A(s,a))$$

Restar la media es un truco de identificabilidad (si no, V y A no quedan únicos). La ganancia: el agente **actualiza V(s) en cada paso aunque no haya probado todas las acciones**, así aprende rápido qué estados son malos.

5.3 Prioritized Experience Replay (PER) — estudia lo difícil más veces

Problema: el agente guarda sus experiencias en un buffer y reaprende de ellas. Si las muestrea al azar, gasta tiempo en transiciones que ya domina.

Solución: muestrear más las transiciones donde más se equivocó (mayor TD-error $|\delta|$).

$$\begin{aligned} \text{prioridad: } p_i &= |\delta_i| + \epsilon \\ \text{probabilidad: } P(i) &= p_i^\alpha / \sum p_k^\alpha \quad (\alpha=0.6: \text{mezcla entre azar y prioridad}) \end{aligned}$$

Pero priorizar **sesga** el aprendizaje (ves lo difícil más de lo real). Se corrige con **pesos de importance sampling**:

$$w_i = (1 / (N \cdot P(i)))^\beta \quad \text{con } \beta \text{ subiendo de } 0.4 \rightarrow 1.0$$

β crece durante el entrenamiento porque la corrección importa más al final, cuando la política ya casi converge.

5.4 n-step returns — mejor asignación de crédito

Problema: el DQN de 1 paso propaga la información de recompensa muy lento (un paso por actualización).

Solución: mirar $N=3$ pasos reales antes de estimar. El objetivo combina 3 recompensas reales más la estimación a 3 pasos:

$$y = \underbrace{r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2}}_{\text{recompensas reales}} + \underbrace{\gamma^3 \cdot Q(s_{t+3}, \dots)}_{\text{bootstrap descontado}}$$

Detalle técnico que defiende correctitud: el bootstrap se descuenta por γ^N (no por γ). Tuvimos un bug donde usábamos γ y lo corregimos. Si el episodio termina antes de los N pasos, se corta y el factor $(1-d)$ anula el bootstrap.

6. Cómo se entrena (el bucle)

Lo que minimiza la red es el error entre su predicción y el objetivo y :

$$\begin{aligned} \text{TD-error: } \delta &= y - Q(s, a; \theta) \\ \text{Pérdida: } L(\theta) &= (1/B) \sum w_i \cdot \text{Huber}(\delta_i) \end{aligned}$$

- **Huber** (smooth L1): se comporta como cuadrático cerca de 0 y lineal lejos. Robusto a outliers, evita que un TD-error gigante desestabilice el entrenamiento.
- w_i : los pesos IS del PER.
- Optimizador **Adam**, lr 5×10^{-4} con recocido coseno, gradiente recortado a norma 10.
- La red target θ^- se **sincroniza con θ cada 2.000 pasos** (objetivo estable).

Exploración: ϵ -greedy

El agente explora al azar con probabilidad ϵ , si no, explota su mejor acción. ϵ baja **linealmente de 1.0 a 0.05 en 50.000 pasos**: al principio prueba de todo, al final casi siempre usa lo aprendido.

El modo torreta (máscara de acciones)

En la sala (`defend_the_center`) no queremos que avance. En vez de reentrenar, **enmascaramos** las acciones de avanzar: ponemos su $Q = -\infty$ antes del argmax . El agente se queda quieto girando y disparando, como una torreta, reutilizando la red ya entrenada.

7. La recompensa (reward shaping)

Por qué la diseñamos a mano

El reward que trae ViZDoom premia **velocidad de avance**. El agente aprendió a explotar eso: se movía mucho para farmear puntos sin completar el nivel. Lo descartamos y construimos una recompensa propia.

Jerarquía (de mayor a menor)

EVENTO	VALOR	POR QUÉ
Completar nivel (pasillo)	+300	Domina todo: el objetivo real
Matar enemigo	+150	Objetivo principal
Morir	-100	Sin esto, "lanzarse y morir" salía rentable
Sobrevivir ronda (sala)	+50	La "meta" equivalente en la sala
Disparar a la nada	-5	Cuida munición, rompe el "rociar paredes"
Disparar con blanco	+2	Señal densa que guía al combate
Esquivar con enemigo	+0.8	Fomenta evasión
Daño recibido	$-0.8 \cdot \Delta \text{vida}$	Enseña a cubrirse
Progreso (pasillo)	$+0.2 \cdot \Delta x$	Solo terreno nuevo (no farmeable)
Encarar / barrer (sala)	+0.2 / +0.05	Apuntar y escanear

La idea del "progreso no farmeable"

En vez de premiar la velocidad, premiamos **solo el avance monótono**: cada unidad de terreno nuevo hacia la meta, una sola vez (delta de la X máxima alcanzada). Así ir y venir no acumula puntos, y el óptimo de la recompensa coincide con **completar el nivel**.

Honestidad técnica (por si preguntan): esto se parece al *potential-based reward shaping* de Ng et al. (1999), pero no es exactamente la fórmula $F = \gamma \Phi(s') - \Phi(s)$. Es un premio por nuevo máximo de posición. Si te lo preguntan, di que la **motivación** es la misma (no alterar la política óptima, solo guiarla) aunque la implementación es más simple.

8. Resultados

- **Detector:** mAP@0.5 = 0.908 en validación.
- **Agente (sala):** la recompensa media móvil (20 episodios) sube de ~300 a ~1.400. Eso es aprendizaje real, no ruido: la curva tiene tendencia clara hacia arriba.
- La corrida final corre en Kaggle (GPU) hasta 1.000.000 de pasos.

Por qué tarda (si lo preguntan)

Cada paso ejecuta una inferencia de YOLO, así que es ~10× más lento por paso que un agente que use solo las variables del juego. Es el precio de tener percepción visual real. 1M de pasos ≈ 9-10 horas en GPU.

9. Banco de preguntas de sustentación

Prepáralas. Son las que probablemente caigan.

P: ¿Por qué Double DQN y no DQN normal? R: El DQN sobreestima los valores porque usa el mismo max para elegir y evaluar. Double separa esas dos funciones en dos redes y corrige el sesgo.

P: ¿Qué es la arquitectura Dueling y qué aporta? R: Separa Q en valor del estado $V(s)$ y ventaja $A(s,a)$. Permite aprender qué estados son buenos sin tener que probar todas las acciones, lo que acelera el aprendizaje.

P: ¿Qué problema resuelve el Prioritized Replay? R: Que muestrear experiencias al azar desperdicia tiempo en lo ya aprendido. PER prioriza las transiciones con mayor error, y corrige el sesgo resultante con pesos de importancia.

P: ¿Por qué n-step y no 1-step? R: n-step propaga la señal de recompensa más rápido y da crédito más preciso a secuencias de acciones. Usamos $N=3$.

P: ¿Por qué no aprenden directo de los píxeles? R: Eficiencia e interpretabilidad. Sobre píxeles necesitaríamos millones de frames y una CNN profunda. Con YOLO de por medio, el agente aprende sobre 39 features con significado.

P: ¿Cómo evitan que el detector "alucine"? R: Dataset in-domain auto-etiquetado desde el motor (sin brecha de dominio), umbral de confianza alto (0.40), y para la recompensa un umbral aún más estricto (0.50). Además solo confiamos en las 4 clases fiables según la matriz de confusión.

P: ¿Cómo etiquetaron el dataset? R: No a mano. ViZDoom expone un `labels_buffer` con las cajas ground-truth, así que el etiquetado fue automático.

P: ¿Qué significa el factor γ ? R: El descuento. $\gamma=0.99$ hace al agente paciente, valora recompensas futuras casi como las presentes. Necesario porque en Doom el premio llega con retraso.

P: ¿Por qué diseñaron la recompensa a mano? R: El reward nativo premiaba velocidad y el agente lo farneaba sin completar el nivel. La recompensa propia alinea el incentivo con el objetivo real (matar,

sobrevivir, llegar).

P: ¿Qué es el modo torreta? R: Enmascarar las acciones de avanzar ($Q = -\infty$) en la sala, para que el agente solo gire y dispare. Reutiliza la red sin reentrenar.

P: ¿Cuál es la diferencia entre los dos escenarios? R: `deadly_corridor` es un pasillo: hay que avanzar a la meta (recompensa de progreso + meta +300). `defend_the_center` es una sala: hay que sobrevivir girando (bono de supervivencia + apuntar + escanear).

P: ¿Qué harían con más tiempo? R: Ampliar el dataset a más clases, comparar contra un baseline de DQN sobre píxeles para medir cuánto aporta la percepción explícita, y extender los pasos de entrenamiento.

10. Guion de exposición (5 minutos)

1. (30 s) Frase de apertura de la sección 0.
 2. (1 min) El mapa: percepción + política, y por qué separarlas (no aprender de píxeles).
 3. (1 min) Percepción: YOLO + el truco del auto-etiquetado in-domain + mAP 0.908.
 4. (1.5 min) Política: MDP, Q, Bellman, y las 4 mejoras (problema → solución de cada una).
 5. (45 s) Recompensa diseñada a mano y el problema del farreo.
 6. (15 s) Resultados: la curva sube de 300 a 1.400 → el agente aprende. Cierre.
-

Relacionado

- sustentacion_proyecto_doom — diapositivas en PDF para proyectar
- Código: `src/policy/rl_agent.py` , `src/policy/dqn.py` , `src/env/rl_env.py`
- Notebook de entrega: `doom_entrega.ipynb` (idéntico a `doom-tam.ipynb`)